

System Design (HLD) – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 4 Jan 2027 – 17 Jan 2027 (Days 141–154)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

High-level design framework, estimation, full system designs, CAP theorem, and interview preparation.

⚠️ PREREQUISITES – Complete FIRST (strict order)

Order	PDF
1	<code>10b-networking-http-fundamentals-complete.pdf</code>
2	<code>06-databases-complete.pdf</code> (scaling, replication)
3	<code>10c-system-design-prerequisites-complete.pdf</code> (caching, LB, queues)

When: Month 6, Days 144+ (after 10c).

Before distributed deep dive: Complete URL shortener + feed designs in this PDF first.

Table of Contents

1. HLD Interview Framework
2. Back-of-Envelope Estimation

3. CAP Theorem & Consistency
 4. URL Shortener
 5. News Feed / Timeline
 6. Chat System
 7. Notification System
 8. E-Commerce Platform
 9. Payment System
 10. Uber / Ride-Sharing
 11. Scaling Patterns
 12. Interview Q&A
 13. ByteByteGo & Resources
 14. Product-company HLD (1 CR)
 15. YouTube / Netflix
 16. Google Drive / Dropbox
 17. Typeahead
 18. Ticketmaster / seat inventory
-

1. HLD Interview Framework

RADIO Framework (45-60 min)

R – Requirements (5 min)
Functional + Non-functional
Out of scope

A – Architecture (10 min)
High-level diagram
Core components

D – Data Model (10 min)
Entities, relationships
Database choice + schema

I – Interface (5 min)
Key API endpoints
Request/response examples

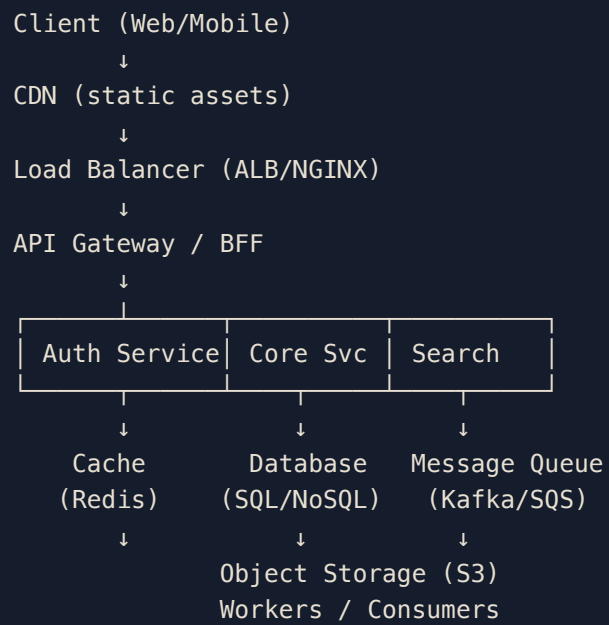
O – Optimizations (15 min)
Bottlenecks, scaling
Trade-offs, deep dives

Requirements Checklist

Functional: What features must the system support? **Non-Functional:**

- Scale (DAU, QPS, data size)
- Latency (p99 targets)
- Availability (99.9%? 99.99%?)
- Consistency (strong vs eventual)
- Durability

Standard Architecture Layers



2. Back-of-Envelope Estimation

Useful Numbers (2024)

Metric	Value
1 day	86400 seconds $\approx 10^5$ s
1 year	$\sim 3 \times 10^7$ seconds
1 KB	10^3 bytes
1 MB	10^6 bytes
1 GB	10^9 bytes
1 TB	10^{12} bytes
Read 1 MB from memory	~ 0.25 ms
Read 1 MB from SSD	~ 1 ms
Read 1 MB from HDD	~ 10 ms
Network round trip (same datacenter)	~ 0.5 ms
Network round trip (cross continent)	~ 150 ms

Example: Twitter-scale

Assumptions:

- 500M DAU
- Each user: 2 tweets/day read, 0.5 tweets/day write
- Average tweet: 300 bytes

Write QPS:

$500M \times 0.5 / 86400 \approx 3000$ writes/sec
Peak (3x): ~ 9000 writes/sec

Read QPS:

$500M \times 2 / 86400 \approx 12,000$ reads/sec
Peak: $\sim 36,000$ reads/sec

Storage (5 years):

$500M \times 0.5 \times 365 \times 5 \times 300$ bytes ≈ 140 TB

QPS Formula

$$\text{QPS} = (\text{DAU} \times \text{actions_per_user_per_day}) / 86400$$

Peak QPS $\approx$ 2–3 $\times$ average QPS

3. CAP Theorem & Consistency

CAP Theorem

In a partition, choose **Consistency** OR **Availability** (Partition tolerance is mandatory in distributed systems).

System	Choice	Example
CP	Consistency over Availability	Banking, inventory
AP	Availability over Consistency	Social feeds, likes count
CA	Only on single node (not distributed)	Single PostgreSQL

Consistency Models

Model	Description	Use Case
Strong	Read sees latest write	Bank balance
Eventual	Replicas converge over time	DNS, social likes
Causal	Related ops seen in order	Chat messages
Read-your-writes	User sees own writes	Profile update

Database Selection Guide

Need	Choose
ACID transactions	PostgreSQL, MySQL
Flexible schema, horizontal scale	MongoDB, DynamoDB
Graph relationships	Neo4j
Time-series metrics	InfluxDB, TimescaleDB
Full-text search	Elasticsearch
Caching	Redis, Memcached
Analytics	Snowflake, BigQuery

4. URL Shortener

Requirements

Functional:

- Shorten long URL → short code
- Redirect short URL → original
- Optional custom alias, expiration, analytics

Non-Functional:

- 100M URLs/day write, 10:1 read/write ratio
- Low latency redirect (< 100ms)
- High availability

High-Level Design

```
Client → LB → API Servers → Redis (cache) → DB
                        ↓
                    Base62 encode / hash
```

API Design

```
POST /api/v1/urls
Body: { "longUrl": "https://...", "customAlias": "optional", "expiresIn": 86400 }
Response: { "shortUrl": "https://short.ly/abc123", "shortCode": "abc123" }
```

```
GET /abc123 → 301 Redirect to long URL
GET /api/v1/urls/:code/stats → click analytics
```

Data Model

```
-- PostgreSQL
CREATE TABLE urls (
  id          BIGSERIAL PRIMARY KEY,
  short_code  VARCHAR(10) UNIQUE NOT NULL,
  long_url    TEXT NOT NULL,
  user_id     BIGINT,
  created_at  TIMESTAMP DEFAULT NOW(),
  expires_at  TIMESTAMP,
  click_count BIGINT DEFAULT 0
);
CREATE INDEX idx_short_code ON urls(short_code);
```

Short Code Generation

Option 1: Auto-increment ID → Base62 encode (a-zA-Z0-9) **Option 2:** MD5 hash of URL → take first 7 chars (collision handling needed) **Option 3:** Pre-generate codes in range, assign on demand

```
const BASE62 = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

function encodeBase62(num: bigint): string {
  if (num === 0n) return "0";
  let result = "";
  while (num > 0n) {
    result = BASE62[Number(num % 62n)] + result;
    num /= 62n;
  }
  return result;
}

// 7 chars Base62 = 62^7 ≈ 3.5 trillion URLs
```

Scaling

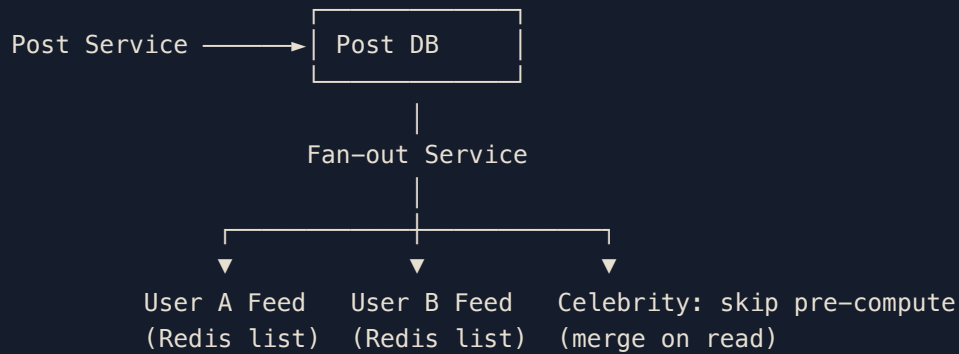
- **Cache:** Redis — 80/20 rule, cache hot URLs
- **Read replicas:** Redirect is read-heavy
- **DB sharding:** Shard by short_code hash
- **CDN/Edge:** Cache 301 redirects at edge (careful with analytics)
- **Rate limiting:** Prevent abuse

5. News Feed / Timeline

Requirements

- Users post tweets/status updates
- Home feed: aggregated posts from followed users
- Fan-out on write vs fan-out on read
- Support celebrities (millions of followers)

Architecture



Fan-Out Strategies

Strategy	Pros	Cons
Fan-out on write	Fast reads	Slow writes for celebrities
Fan-out on read	Fast writes	Slow reads for users with many follows
Hybrid	Best of both	More complex

Hybrid approach:

- Normal users (< 10K followers): fan-out on write
- Celebrities: fan-out on read — merge celebrity posts at read time

API Design

```
POST /api/v1/posts
Body: { "content": "Hello world", "mediaUrls": [] }

GET /api/v1/feed?cursor=abc&limit=20
Response: { "posts": [...], "nextCursor": "def" }
```

Data Model

```
posts: { postId, userId, content, createdAt, mediaUrls[] }  
follows: { followerId, followeeId, createdAt }  
feed:{userId}: [postId1, postId2, ...] (Redis sorted set by timestamp)
```

6. Chat System

Requirements

- 1:1 and group messaging
- Real-time delivery
- Online presence
- Message history, read receipts
- 50M DAU, 100 msg/user/day

Architecture

```
Mobile/Web  
  ↓ WebSocket  
Chat Gateway (sticky sessions)  
  ↓  
Message Service → Message Store (Cassandra)  
  ↓  
Message Queue (Kafka)  
  ↓  
Push Notification Service  
Presence Service (Redis)
```

WebSocket vs Long Polling

Method	Latency	Complexity	Use
WebSocket	Lowest	Medium	Chat, gaming
SSE	Low (one-way)	Low	Live feeds
Long polling	Higher	Low	Fallback

API Design

```
WS /ws?token=jwt
  → send: { type: "message", to: "userId", content: "hi" }
  → recv: { type: "message", from: "userId", content: "hi", ts: 123 }

GET /api/v1/conversations
GET /api/v1/conversations/:id/messages?before=ts&limit=50
POST /api/v1/conversations (create group)
```

Data Model

```
messages: { msgId, conversationId, senderId, content, timestamp, status }
conversations: { convId, type: direct|group, participants[], lastMessageAt }
user_presence: Redis { userId → { status, lastSeen } }
```

Group Chat Fan-out

- Message written once to store
- Kafka topic per conversation OR push to online participants via gateway registry
- Gateway maintains: userId → websocket connection mapping (Redis)

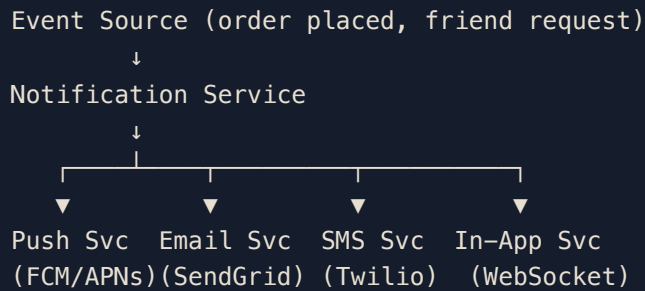
7. Notification System

Requirements

- Multi-channel: push, email, SMS, in-app

- Priority levels
- User preferences (opt-out)
- 1B notifications/day
- At-least-once delivery

Architecture



API Design

```
POST /api/v1/notifications (internal)
Body: {
  "userId": "123",
  "type": "order_shipped",
  "channels": ["push", "email"],
  "payload": { "orderId": "456", "title": "...", "body": "..." }
}

GET /api/v1/notifications?unread=true
PATCH /api/v1/notifications/:id/read
PUT /api/v1/users/me/notification-preferences
```

Key Design Decisions

- **Queue-based:** Decouple producers from delivery; handle spikes
- **Idempotency:** Dedupe key prevents duplicate notifications
- **Retry with backoff:** Failed SMS/email retried 3×
- **Rate limiting:** Max 10 push/hour per user
- **Template engine:** Dynamic content per channel

Data Model

```
notifications: { id, userId, type, channels[], payload, status, createdAt }
preferences: { userId, channel, type, enabled }
delivery_log: { notificationId, channel, status, attempts, deliveredAt }
```

8. E-Commerce Platform

Requirements

- Product catalog, search, cart, checkout, orders
- Inventory management
- Payment integration
- 10M users, 100K orders/day

Architecture

```

      [ CDN / S3 ] (product images)

Client → LB → API Gateway
               |
               |— Product Service → PostgreSQL + Elasticsearch
               |— Cart Service → Redis
               |— Order Service → PostgreSQL
               |— Inventory Service → PostgreSQL (with locking)
               |— Payment Service → External gateway
               |— Search Service → Elasticsearch
               ↓
             Kafka (order events)
               ↓
      Email / Analytics / Warehouse
```

API Design

```
GET /api/v1/products?category=electronics&page=1
GET /api/v1/products/:id
GET /api/v1/search?q=laptop

POST /api/v1/cart/items      { productId, quantity }
GET /api/v1/cart
POST /api/v1/orders         { cartId, shippingAddress, paymentMethod }
GET /api/v1/orders/:id
```

Data Model

```
products (id, name, description, price, category_id, inventory_count)
orders (id, user_id, status, total, created_at)
order_items (order_id, product_id, quantity, price_at_purchase)
cart (user_id, items JSONB) -- Redis with TTL
```

Critical Flow: Place Order

1. Validate cart items + prices
2. Reserve inventory (optimistic/pessimistic lock)
3. Create order (status: PENDING)
4. Process payment
5. On success: confirm order, decrement inventory, emit event
6. On failure: release inventory, mark order FAILED

Saga pattern for distributed transaction across inventory + payment + order.

9. Payment System

Requirements

- Process credit card, UPI, wallet payments
- Idempotent transactions
- PCI DSS compliance (never store raw card data)

- Reconciliation, refunds

Architecture

```
Client → Payment API → Payment Gateway (Stripe/Razorpay tokenization)
      ↓
    Payment Processor
      ↓
  Ledger Service (double-entry)
      ↓
    PostgreSQL (transactions, ledger)
      ↓
  Reconciliation Worker (daily)
```

API Design

```
POST /api/v1/payments
Headers: Idempotency-Key: uuid
Body: {
  "orderId": "ord_123",
  "amount": 99900,
  "currency": "INR",
  "paymentMethodId": "pm_token_from_stripe"
}
Response: { "paymentId": "pay_456", "status": "succeeded" }

POST /api/v1/payments/:id/refund
Body: { "amount": 99900, "reason": "customer_request" }
```

Double-Entry Ledger

Every transaction creates two entries:

```
Debit: Customer Wallet    -₹999
Credit: Merchant Account  +₹999
```

Ensures books always balance. Audit trail for compliance.

Idempotency

```
// Store idempotency key → response mapping for 24 hours
async function processPayment(idempotencyKey: string, payload: PaymentRequest) {
  const cached = await redis.get(`idem:${idempotencyKey}`);
  if (cached) return JSON.parse(cached);

  const result = await gateway.charge(payload);
  await redis.setex(`idem:${idempotencyKey}`, 86400, JSON.stringify(result));
  return result;
}
```

Security

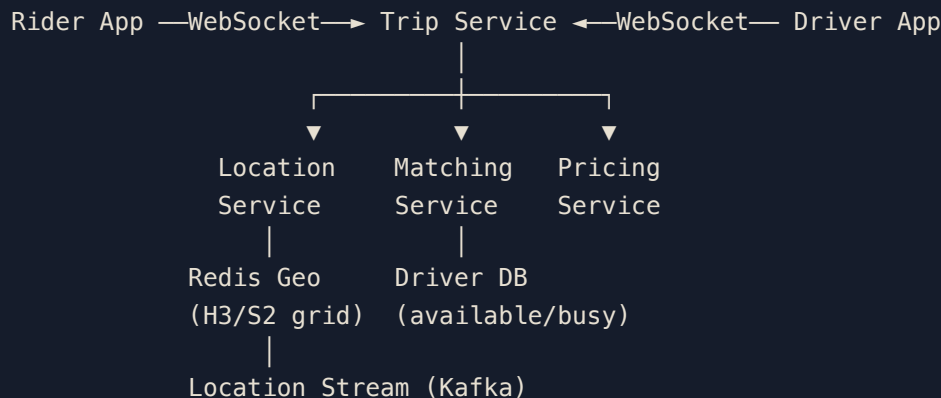
- Tokenize cards via Stripe/Razorpay — never touch PAN
- TLS everywhere, encrypt PII at rest
- Webhook signature verification
- Fraud detection rules engine

10. Uber / Ride-Sharing

Requirements

- Riders request rides, drivers accept
- Real-time location tracking
- Matching rider to nearest available driver
- ETA, fare estimation, trip history
- 10M rides/day

Architecture



API Design

```
POST /api/v1/trips/request
Body: { "pickupLat": 12.97, "pickupLng": 77.59, "dropLat": ..., "dropLng": ... }
Response: { "tripId": "...", "estimatedFare": 250, "eta": 5 }

POST /api/v1/trips/:id/accept    (driver)
POST /api/v1/trips/:id/start
POST /api/v1/trips/:id/complete

WS location updates: { "lat": 12.97, "lng": 77.59, "heading": 90 }
GET /api/v1/trips/:id/track    (rider sees driver location)
```

Matching Algorithm

1. Rider requests trip at (lat, lng)
2. Query geospatial index for drivers within 5km radius
3. Filter: status=available, vehicle type matches
4. Rank by: distance, rating, acceptance rate
5. Send request to top 3 drivers sequentially (30s timeout each)
6. First accept wins – notify others trip taken

Geospatial indexing: Redis GEO, Google S2, Uber H3 hexagonal grid.

Data Model

```
trips: { tripId, riderId, driverId, pickup, dropoff, status, fare, timestamps }  
drivers: { driverId, location, status, vehicleType, rating }  
locations: Redis GEOADD drivers:geo lng lat driverId
```

Surge Pricing

- Divide city into zones (H3 cells)
- Track demand (requests) vs supply (available drivers) per zone
- multiplier = $f(\text{demand/supply ratio})$
- Update every 5 minutes

11. Scaling Patterns

Horizontal vs Vertical

Vertical	Horizontal
Bigger machine	More machines
Simple, limited ceiling	Complex, unlimited scale
Good for DB initially	Good for stateless services

Caching Strategies

Pattern	Description
Cache-aside	App reads cache, on miss reads DB, writes cache
Write-through	Write to cache + DB simultaneously
Write-behind	Write cache, async flush to DB

Load Balancing Algorithms

- Round Robin
- Least Connections
- IP Hash (session stickiness)
- Weighted Round Robin

Database Scaling

```
Single DB → Read Replicas → Sharding → Federation
          ↓
        Caching (Redis)
          ↓
        CQRS (separate read/write models)
```

Message Queue Use Cases

- Async processing (send email after order)
- Decoupling services
- Peak load buffering
- Event sourcing / audit log

12. Interview Q&A

Q: How do you handle hot keys in Redis? A: Local cache in app, replicate key across shards, read replicas, or split hot key into multiple sub-keys.

Q: SQL vs NoSQL? A: SQL for ACID, complex joins, structured data. NoSQL for flexible schema, horizontal scale, high write throughput.

Q: What is consistent hashing? A: Distributes keys across nodes; adding/removing node only remaps K/n keys. Used in distributed caches and sharding.

Q: How to design for 99.99% availability? A: Multi-AZ deployment, no single point of failure, health checks, auto-failover, circuit breakers, graceful degradation.

Q: Explain CDN. A: Edge servers cache static content close to users. Reduces latency and origin load. Invalidation strategy matters.

Q: How does Kafka help? A: Durable, ordered, replayable event log. Decouples producers/consumers, handles traffic spikes, enables event-driven architecture.

Q: What is API Gateway? A: Single entry point for routing, auth, rate limiting, SSL termination, request aggregation.

Q: How to prevent double booking? A: Pessimistic locking, optimistic locking with version, distributed lock (Redis), or serializable transaction isolation.

Q: Eventual consistency example? A: Like count on a post — doesn't need to be exact in real-time; async counter aggregation is fine.

Q: Microservices vs Monolith? A: Monolith simpler early; microservices for independent scaling, team autonomy, tech diversity — at cost of complexity.

13. ByteByteGo & Resources

ByteByteGo

Resource	URL	Content
Newsletter	bytebytego.com	Weekly system design
YouTube	youtube.com/@ByteByteGo	Visual explanations
Courses	bytebytego.com/courses	Structured learning

Recommended Reading Order

1. Scalability basics (LB, cache, CDN, DB replication)
2. URL Shortener (simplest complete design)
3. News Feed (fan-out trade-offs)
4. Chat (WebSocket, presence)
5. Notification (async, multi-channel)
6. E-Commerce (inventory, saga)
7. Payment (idempotency, ledger)
8. Uber (geospatial, real-time matching)

Other Resources

Resource	Focus
Designing Data-Intensive Applications (Kleppmann)	Deep fundamentals
System Design Interview (Alex Xu)	Vol 1 & 2
Grokking System Design	Interactive
High Scalability Blog	Real-world architectures

Practice Schedule (8 Weeks)

Week	Topic	Deliverable
1	Framework + estimation	5 estimation drills
2	URL Shortener + Paste bin	Full design doc
3	News Feed + Twitter	Fan-out analysis
4	Chat + WhatsApp	WebSocket deep dive
5	Notification + Email	Queue design
6	E-Commerce + Amazon	Order flow saga
7	Payment + Stripe	Idempotency + ledger
8	Uber + Lyft	Geospatial matching

Quick Reference Card

Framework: Requirements → Architecture → Data → API → Optimize
Estimate: $DAU \times \text{actions} / 86400 = \text{QPS}$; peak = 2-3×
CAP: Partition → choose C or A
Scale: CDN → LB → Cache → Replicas → Shards → Queue
Always: State assumptions, draw diagram, discuss trade-offs

Remember: There's no single correct answer. Communicate trade-offs, justify choices, and iterate when interviewer adds constraints.

14. Product-company HLD (1 CR)

Google / Meta / Uber / Atlassian grade **product judgment**, not a Kafka diagram. Full framework:

`20-1cr-product-company-complete.pdf` §5.

Open every design with: who is the user, what is the ONE metric, what is MVP vs v2.

Put Kafka / K8s only on the async or deploy slide — never as sentence one.

Product RADIO (short):

1. Product + metric (3 min)
2. Requirements + out of scope (5)
3. API JSON (5)
4. Data model + keys (7)
5. Happy path (8)
6. Scale the hottest QPS (10)
7. User-visible failure (5)

1 CR extra designs (Phase 7F): YouTube, Drive, Typeahead, Ticketmaster, plus Search / Docs / Crawler in Phase 9.

15. YouTube / Netflix

Product: watch a video with low start time. Metric: start-play p99 < 2s.

MVP: upload → process → play. Out of scope: live, ads (unless pushed).

API: `POST /videos` (returns upload URL) • `GET /videos/:id` • `GET /videos/:id/manifest.m3u8`

Data: `videos(id, owner_id, status, duration, playback_url)` • object store for chunks • CDN in front.

Architecture:

Client → CDN → origin (object store)

Upload → API → object store → queue → transcode workers → write renditions (360/720/1080)

Deep dive: ABR (client picks bitrate). Thumbnails + captions = more async jobs.

Recommendations (v2): offline batch → feature store → online cache. **Do not JOIN** similar videos on the play request.

Failure: if transcode lags, play original + “Processing HD”. If CDN miss storm, origin shield.

Numbers: 10M DAU $\times$ 5 videos $\times$ 800 MB avg $\rightarrow$ storage + CDN egress dominate cost, not the API box.

16. Google Drive / Dropbox

Product: sync files across devices. Metric: conflict rate + upload success.

MVP: upload, download, list folder, share link.

API: PUT /files/:id/chunks/:n • POST /files/:id/commit • GET /fs?parent= • POST /share

Data: metadata DB (file_id, parent, name, version, checksum) + chunk store keyed by hash (dedup).

Sync: client hashes 4 MB chunks $\rightarrow$ upload missing hashes $\rightarrow$ commit new version.

Conflict: MVP last-writer-wins + keep previous version. v2: branch + notify both clients.

Notify: websocket / push on file_id watchers — not polling every file.

Failure: metadata DB is source of truth; chunk store is immutable. Retry commit is idempotent on version .

17. Typeahead

Product: suggest queries as you type. Metric: p99 < 50ms.

API: GET /suggest?q=ho $\rightarrow$ [{text, score}]

Data: prefix index (trie or prefix-partitioned posting). Value = query + popularity.

Hot path:

Client $\rightarrow$ edge cache (hot prefixes "a","in","how") $\rightarrow$ suggest service (in-memory index)

Rank: frequency $\times$ recency (offline). Personalization = blend last 20 user queries (v2).

Failure: ranker down $\rightarrow$ serve cached global popular. Empty query $\rightarrow$ trending list.

Follow-up: typos $\rightarrow$ fuzzy / BK-tree or “did you mean” from edit-distance 1 on a small dictionary — not on the whole corpus at request time.

18. Ticketmaster / seat inventory

Product: do not double-sell a seat. Metric: zero oversell, hold expiry 8 min.

API: POST /holds {event, seats[]} → hold_id • POST /checkout {hold_id, payment} • DELETE /holds/:id

Data: seats(event_id, seat_id, status, hold_id, hold_until) • unique (event_id, seat_id) .

Race:

1. Transaction: SELECT ... FOR UPDATE (or Redis lock per seat)
2. If available → status=held , hold_until=now+8m
3. Payment webhook → status=sold (idempotent on payment_id)
4. Sweeper releases expired holds

Do not “check then set” without a lock. Kafka is optional for email; it is **not** the inventory source of truth.

Failure: payment succeeds twice → same payment_id no-ops. Payment fails → hold expires, seat returns.

Quick Reference Card (product)

Open with: user + metric + MVP

Then: API JSON → keys/indexes → happy path → hottest QPS → user-visible fallback

Kafka/K8s: only if async or deploy is the bottleneck

1 CR set: Feed, Chat, Uber, Payment, YouTube, Drive, Typeahead, Tickets

System Design Interview – 100 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers High Level Design (HLD), scalability, databases, caching, and classic system design problems for product company interviews. Spell out abbreviations on first use.

1. What is System Design in the context of product company interviews?

System Design is the process of defining architecture for a large-scale application: how users interact with it, which services and databases you use, how data flows, and how the system handles growth and failures. Interviewers evaluate your ability to trade off consistency, availability, latency, cost, and complexity.

2. What are the key steps in a system design interview?

1. Clarify functional and non-functional requirements.
- 2) Estimate scale (users, queries per second, storage).
- 3) Propose High Level Design (APIs, main components).
- 4) Deep-dive data model and critical paths.
- 5) Address bottlenecks, scaling, failure modes.
- 6) Summarize trade-offs. Always think aloud and collaborate.

3. How would you design a URL shortener like bit.ly?

Functional: shorten long URL, redirect short URL, optional analytics. API: `POST /urls {longUrl}` → `{shortUrl}`, `GET /{shortCode}` → 302 redirect. Generate short code via base-62 encoding of auto-increment ID or hash (handle collisions). Store mapping in database (SQL or NoSQL). Cache hot URLs in Redis. Scale reads with replicas and CDN; writes with sharded database. Discuss custom aliases and expiration.

4. How do you generate unique short codes for a URL shortener?

Option A: Auto-increment ID encoded in base-62 (a-z, A-Z, 0-9)—guaranteed unique, predictable length growth. Option B: Random 6–8 character string—check uniqueness in database; retry on

collision. Option C: Hash (MD5/SHA) truncated—risk of collision; use longer codes or salting. For interviews, base-62 of distributed ID (Snowflake) is a strong answer.

5. How would you design Twitter (now X) at a high level?

Core features: post tweet, follow users, home timeline, search. Services: User Service, Tweet Service, Timeline Service, Search Service, Notification Service. Write path: tweet saved to Tweet DB, fan-out to followers' timelines (push model) or merged at read (pull model). Hybrid fan-out for celebrities (millions of followers). Cache timelines in Redis. Use Kafka for async fan-out.

6. Explain fan-out on write vs fan-out on read for timelines.

Fan-out on write (push): when user posts, precompute and write tweet ID to each follower's timeline cache—fast reads, slow/expensive writes for popular users. Fan-out on read (pull): store tweets; at read time merge tweets from followed users—cheap writes, slower reads. Hybrid: push for normal users, pull for celebrities.

7. How would you design Instagram?

Features: upload photo/video, follow, feed, likes, comments, stories. Upload flow: client → object storage (Amazon S3) via pre-signed URL; metadata in database. Feed similar to Twitter fan-out. CDN serves images globally. Separate services for media processing (thumbnails, transcoding), social graph, and notifications.

8. What is the difference between SQL and NoSQL for system design?

SQL (Structured Query Language) databases (PostgreSQL, MySQL) offer strong schemas, joins, and ACID transactions—good for relational data (orders, payments). NoSQL (MongoDB, Cassandra, DynamoDB) offers flexible schemas and horizontal scaling—good for high write throughput, simple access patterns, or document/key-value models. Many systems use both (polyglot persistence).

9. When would you choose a document database vs a wide-column store?

Document DB (MongoDB): flexible JSON documents, rich queries, good for user profiles, content catalogs. Wide-column (Cassandra, HBase): optimized for time-series and high write volume with partition key access—good for feeds, metrics, message logs. Choose based on access pattern, not hype.

10. What is a Content Delivery Network (CDN)?

A CDN is a geographically distributed network of edge servers that cache static content (images, videos, JavaScript, CSS) close to users. Reduces latency and origin server load. Examples: CloudFront, Cloudflare, Akamai. Use CDN for media-heavy apps like Instagram and Netflix.

11. How do you design APIs for a large system?

Prefer REST for resource-oriented CRUD or GraphQL when clients need flexible field selection. Define clear endpoints, versioning (`/v1/`), authentication (OAuth 2.0 tokens), rate limiting, and pagination (cursor-based for large feeds). Use idempotency keys for payment-like operations.

12. What is horizontal vs vertical scaling?

Vertical scaling (scale up) adds CPU/RAM to one machine—simple but has limits. Horizontal scaling (scale out) adds more machines—requires load balancing and distributed data. Large product systems almost always scale horizontally.

13. What is database sharding?

Sharding splits data across multiple database instances by a shard key (e.g., `user_id % N`). Each shard holds a subset of data. Enables write scalability beyond single-node limits. Challenges: cross-shard queries, rebalancing, hot shards. Consistent hashing helps minimize data movement on resharding.

14. What is replication in databases?

Replication copies data from a primary (leader) node to one or more replica (follower) nodes. Used for read scaling and high availability. Synchronous replication waits for replica ack—stronger consistency, higher latency. Asynchronous replication is faster but may lose recent writes on failover.

15. What is the CAP theorem?

CAP theorem states that in a network partition, a distributed system must choose between **Consistency** (all nodes see same data) and **Availability** (every request gets a response). Partition tolerance is mandatory in distributed systems. In practice you tune consistency vs availability per use case (e.g., strong for payments, eventual for likes count).

16. What is eventual consistency?

Eventual consistency means if no new updates occur, all replicas will converge to the same value over time. Acceptable for social likes, view counts, and CDN caches. Unacceptable for bank balances without careful design. Mitigate with version vectors, conflict resolution, or user-facing 'syncing' indicators.

17. How would you design a notification system?

Components: Event Producer (app services) → Message Queue (Kafka) → Notification Service → channels (push, email, SMS). Store user preferences and device tokens. Template engine for message content. Rate limit and batch notifications. Retry with dead-letter queue for failures. Scale workers horizontally.

18. What is a message queue and why use one?

A message queue (Kafka, RabbitMQ, Amazon Simple Queue Service) decouples producers and consumers. Benefits: async processing, load leveling, fault tolerance, and independent scaling. Example: order placed event triggers inventory, shipping, and email services without blocking checkout.

19. How would you design a chat application like WhatsApp?

Real-time delivery via WebSockets or long polling. Message Service stores messages; sync on reconnect. Use sequence numbers per conversation for ordering. Presence service tracks online status. End-to-end encryption for privacy. Push notifications via Apple Push Notification service / Firebase Cloud Messaging when offline.

20. What is WebSocket and when is it used?

WebSocket is a persistent, bidirectional protocol over TCP. Unlike HTTP request/response, server can push data anytime. Used for chat, live sports scores, collaborative editing, and gaming. Fallback to Server-Sent Events or long polling if WebSockets blocked by proxies.

21. How would you design YouTube or a video streaming platform?

Upload: chunked upload to object storage; transcoding pipeline produces multiple resolutions (240p–4K). Metadata and comments in database. CDN delivers video segments (HLS/DASH adaptive bitrate). View counts via async aggregation. Recommendation engine as separate ML pipeline.

22. What is rate limiting at the system design level?

Rate limiting protects services from abuse and overload. Implement at API gateway (token bucket per user/IP). Return HTTP 429 Too Many Requests with Retry-After header. Distributed rate limits need shared store (Redis). Different tiers for free vs premium users.

23. How do you handle authentication in a distributed system?

Use OAuth 2.0 / OpenID Connect: user logs in via identity provider, client gets JWT (JSON Web Token). Services validate JWT signature and expiry statelessly. Refresh tokens for long sessions. Store sessions in Redis for revocation. Never store passwords in plain text—use bcrypt/argon2 hashing.

24. What is an API Gateway?

An API Gateway is a single entry point for clients routing to backend microservices. Handles authentication, rate limiting, SSL termination, request routing, and aggregation. Examples: Kong, AWS API Gateway, Nginx. Simplifies client logic and centralizes cross-cutting concerns.

25. How would you design a search system?

Ingest documents into inverted index (Elasticsearch, Apache Solr). Tokenize, stem, remove stop words. Rank by TF-IDF (Term Frequency-Inverse Document Frequency) or learning-to-rank. Autocomplete via prefix index (Trie or Elasticsearch completion suggester). For Twitter, separate hot index for recent tweets and cold storage for archive.

26. What is Elasticsearch used for?

Elasticsearch is a distributed search and analytics engine built on Apache Lucene. Stores JSON documents, supports full-text search, aggregations, and geo queries. Used for log analytics (ELK stack), product search, and autocomplete. Scale by adding nodes to cluster.

27. How do you design for high availability?

Eliminate single points of failure: multi-Availability Zone deployment, load balancers with health checks, database replication with automatic failover, circuit breakers, graceful degradation (show cached feed if timeline service down). Target SLA (Service Level Agreement) like 99.9% uptime (~8.7 hours downtime/year).

28. What is a circuit breaker pattern?

Circuit breaker stops calling a failing downstream service after threshold failures, returning fallback quickly. States: Closed (normal), Open (fail fast), Half-Open (probe recovery). Prevents cascade failures. Libraries: Resilience4j, Hystrix (legacy). Essential in microservices.

29. What is idempotency and why does it matter?

An operation is idempotent if performing it multiple times has the same effect as once. Critical for retries (network timeouts). Example: payment with idempotency key—duplicate requests do not

double-charge. Implement via unique request ID stored in database before processing.

30. How would you design an e-commerce checkout system?

Flow: cart → inventory check → payment → order creation → notification. Use saga pattern for distributed transaction across inventory, payment, and order services. Reserve inventory temporarily; release on payment timeout. Strong consistency for payment; async for emails.

31. What is the difference between strong and weak consistency?

Strong consistency: after a write, all reads see the latest value immediately (linearizability).

Weak/eventual: reads may return stale data temporarily. Choose strong for financial ledger; eventual for social metrics. Many databases offer tunable consistency levels.

32. How do you estimate storage for a system design question?

Example: 500M users, 2 tweets/user/day, 280 bytes/tweet → 1B tweets/day × 280 B ≈ 280 GB/day raw. Add indexes (~2×), replicas (3×), media separately. Round up. Show structured math—interviewers care about approach, not exact numbers.

33. How do you estimate queries per second (QPS)?

Daily active users × actions per day / 86400 seconds. Example: 100M daily active users, 50 reads/day → 5B reads/day ≈ 58,000 read QPS average; plan for 3–5× peak. Separate read vs write QPS.

34. What is microservices architecture?

Microservices split an application into small, independently deployable services, each owning its data and business logic. Communicate via HTTP/REST or message queues. Benefits: independent scaling, team autonomy, technology diversity. Costs: operational complexity, distributed debugging, network latency.

35. Monolith vs microservices: when to choose which?

Start with modular monolith for new products—simpler ops, faster iteration. Extract microservices when teams scale, components need independent scaling, or deployment cycles conflict. Interview answer: 'I'd start monolith with clear module boundaries, split when metrics justify it.'

36. What is a load balancer and where do you place it?

A load balancer distributes incoming traffic across multiple servers. Layer 4 (transport) routes by IP/port; Layer 7 (application) routes by URL path or headers. Place between clients and app servers, and between app and database read replicas. Algorithms: round robin, least connections, consistent hashing for sticky sessions.

37. How would you design a ride-sharing app like Uber?

Services: User, Driver, Trip, Matching, Pricing, Payment, Notification. Real-time location via GPS updates to geospatial index (Redis Geo, PostGIS). Matching: find nearby available drivers. Surge pricing via demand/supply ratio. Trip state machine: requested → accepted → in-progress → completed.

38. What is geospatial indexing?

Geospatial indexes efficiently query locations within radius or bounding box. Techniques: Geohash, QuadTree, R-tree. Uber uses H3 hexagonal grid. Store driver coordinates updated every few seconds; query 'drivers within 2 km of rider'.

39. How would you design a file storage system like Dropbox?

Client syncs files via chunked upload (deduplicate chunks by hash—content-addressable storage). Metadata (file tree, permissions) in SQL; blobs in object storage (S3). Conflict resolution via versioning or last-write-wins with user notification. Delta sync sends only changed chunks.

40. What is consistent hashing?

Consistent hashing maps keys and nodes to a hash ring. Adding/removing a node only remaps adjacent key range—unlike modulo hashing which remaps almost all keys. Used for distributed

caches, sharding, and load balancing. Virtual nodes improve load distribution.

41. How do you design logging and monitoring?

Centralized logs: app → Fluentd/Logstash → Elasticsearch → Kibana (ELK). Metrics: Prometheus scrapes endpoints, Grafana dashboards. Tracing: OpenTelemetry/Jaeger for request flow. Alert on SLI (Service Level Indicator) breaches: error rate, latency p99, saturation.

42. What is the difference between latency and throughput?

Latency is time for one request to complete (milliseconds). Throughput is requests handled per second. Optimizing one can hurt the other. Batch processing increases throughput but adds latency. Define SLAs (Service Level Agreements) for both (e.g., p99 latency < 200ms, 10K QPS).

43. How would you design a rate-limited public API?

API keys per developer, tiered quotas (1000 req/day free, 1M paid). Enforce at gateway with Redis counters. Return rate limit headers (`X-RateLimit-Remaining`). Document endpoints, provide SDKs, sandbox environment. Billing integration for overages.

44. What is a bloom filter?

A Bloom filter is a space-efficient probabilistic structure testing set membership. May have false positives (says maybe present) but never false negatives. Used to avoid unnecessary database lookups (e.g., 'does this username exist?' or 'is this URL malicious?'). Cannot delete without counting variant.

45. How do you prevent the thundering herd problem?

When cache expires, many requests hit database simultaneously. Solutions: probabilistic early expiration, mutex lock on cache miss (only one thread rebuilds), stale-while-revalidate (serve stale, refresh async), pre-warm cache before peak events.

46. What is database indexing strategy in system design?

Index columns used in WHERE, JOIN, ORDER BY. Composite indexes for multi-column queries (leftmost prefix rule). Avoid over-indexing—slows writes. Use covering indexes to avoid table lookups. For feeds, index `(user_id, created_at DESC)`. Explain trade-off: read speed vs write overhead.

47. How would you design a news feed ranking system?

Candidate generation (follow graph + trending) → feature extraction (recency, engagement, affinity) → ML ranker (gradient boosted trees or neural net) → diversity re-ranking. Train offline on click data; serve online with low latency (<100ms). A/B test ranking changes.

48. What questions should you ask the interviewer in system design?

Clarify scale (users, QPS), read/write ratio, latency requirements, consistency needs, geographic distribution, mobile vs web, and which features are in scope. Asking shows seniority and prevents over-engineering.

49. How do you wrap up a system design interview?

Summarize architecture diagram verbally, highlight key trade-offs (push vs pull timeline, SQL vs NoSQL), mention what you'd monitor, and one future improvement (multi-region, ML personalization). Leave time for interviewer questions.

50. What is caching strategy in system design interviews?

Caching is storing frequently read data in fast storage (Redis, Memcached, or Content Delivery Network) to reduce database load and latency. Patterns: cache-aside (lazy load on miss), write-through (sync write to cache and DB), and TTL-based

expiration. Always discuss cache invalidation, hot keys, and thundering herd mitigation in interviews.

Questions 51-100 (top product companies)

51. How do you estimate queries per second from daily active users?

$DAU \times \text{actions} / 86400 \times \text{peak } 2-3\times$. Say the number. All HLD.

52. When is a Content Delivery Network the first optimization?

Static and video bytes. Origin shield on miss storms. YouTube / Netflix HLD.

53. Cache stampede – how do you stop it?

Lock on miss, probabilistic early expire, or serve stale. Meta / Uber.

54. How do you pick a shard key?

High cardinality, even load, matches the query. Avoid hot celebrity keys. Meta / Uber.

55. What is a hot key in Redis?

One key gets most traffic. Local cache, split, or replicate. Meta.

56. Leader election – why mention it?

One writer for a partition (Kafka, primary DB). etcd/ZooKeeper. Google / Amazon.

57. How do you design unread badges?

Counter per user, increment on write, reset on read, accept eventual. Meta.

58. What is an outbox pattern?

Write DB row + event in one transaction; publisher drains outbox. Payments. Amazon / Stripe.

59. How do you expire paste-bin pastes?

TTL in Redis or a sweep job on `expires_at`. Easy HLD. All.

60. When do you need a message queue vs a database table?

Queue for async fan-out and retries. Table if you must query jobs. Uber / Amazon.

61. How do you design presence (online dots)?

Heartbeat to Redis with TTL, WebSocket disconnect cleanup. WhatsApp HLD. Meta / Uber.

62. What is consistent hashing for?

Caches/shards so adding a node remaps $\sim 1/N$ keys. Amazon / Google.

63. How do you handle multi-region writes?

Single primary region for strong data; or CRDTs/conflict for docs. Google / Meta.

64. Explain CQRS in product language.

Different model for write vs read (feed cache vs post table). Meta / Uber.

65. How do you design a web crawler politely?

Frontier queue, per-host delay, robots.txt, URL dedup. Google / Phase 9.

66. What is an API gateway doing?

Auth, rate limit, routing, TLS. Not your business logic. Adobe / Amazon.

67. How do you store images for an Instagram-like app?

Object store + CDN; metadata in DB; thumbnails async. Meta.

68. Design a rate limiter for 100k requests/s.

Redis cluster, shard by user, local allow-list for health. Uber / Amazon.

69. How do you do search typeahead at Google scale (lite)?

Prefix shards, in-memory, cache first characters, offline rank. Google / PDF #10.

70. What fails if your load balancer is sticky and a node dies?

Those users reconnect; session must live in Redis not memory. Adobe / Uber.

71. How do you design idempotent webhooks inbound?

Store event id, unique constraint, return 200 on replay. Stripe / Razorpay.

72. Explain fan-out-on-write cost for a celebrity.

80M writes per tweet — impossible. Hybrid pull. Meta.

73. How do you keep chat order in a group?

Per-group sequence or vector; display sort by server timestamp + id. Meta / Uber.

74. What is a bloom filter for in shortener/crawler?

Probabilistic “seen URL” to skip disk. Google.

75. How do you design feature flags globally?

SDK + rules service, default false, cache, audit. Meta / LaunchDarkly-style.

76. Object storage vs block vs DB for files?

S3-style for blobs; DB for metadata only. Drive HLD. Google / Dropbox.

77. How do you design an admin analytics dashboard?

Event stream → warehouse (batch); not the OLTP DB. Amazon / Flipkart.

78. What is a dead-letter queue in notifications?

After N fails, park and page a human. Amazon / Uber.

79. How do you avoid thundering herd on cache expire?

Jitter TTLs, lock, stale-while-revalidate. Meta.

80. Design a URL redirect that must be fast.

Cache, 301 vs 302, prefetch DNS. All easy HLD.

81. When is Kafka the wrong answer?

When you need a queryable table or a single-box app. Most junior HLD. PDF #20.

82. How do you model money?

Integer minor units, never float; ledger rows. Stripe / PhonePe.

83. What is a saga vs two-phase commit?

Saga: local tx + compensate. 2PC: locks, rarely at web scale. Amazon / Uber.

84. How do you design seat holds that expire?

hold_until + sweeper + unique seat. Ticketmaster. Flipkart / PDF #10.

85. Explain read-your-writes after posting.

Read primary or cache you just wrote. Meta / Twitter.

86. How do you shard a chat system?

By conversation id so a room lives on one partition. Meta / Slack.

87. What SLI/SLO would you set for a shortener?

Availability 99.9%, p99 redirect < 100 ms. Google SRE-lite. Phase 7G.

88. How do you design logout-all-devices?

Session version in DB; increment; tokens carry version. Amazon / Adobe.

89. CDN purge vs versioned URLs?

Prefer `/img/abc123.jpg` new hash; purge is slow. Google / Netflix.

90. How do you handle clock skew in distributed systems?

Do not trust client clocks; server time; logical clocks if needed. Google.

91. Design a news-letter send to 10M users.

Queue chunks, rate-limit ESP, unsubscribe table, bounce handling. Amazon / Adobe.

92. What is an edge cache vs origin cache?

Edge near user (CDN); origin shield in your region. YouTube HLD.

93. How do you design unique IDs at scale?

Snowflake (timestamp+worker+seq) or UUID v7. Avoid DB autoincrement as the only global id. Uber / Twitter.

94. When do you denormalize like_count?

Read-heavy posts; accept eventual via async increment. Meta.

95. How do you design a document collaborative editor (lite)?

OT or CRDT, persist ops, presence. Google Docs. Phase 9. PDF #20.

96. What is a BFF for mobile vs web?

Different payloads; one gateway per client type. Meta / Atlassian.

97. How do you plan a launch that can rollback?

Feature flag, old binaries, dual-write if schema. Amazon.

98. Explain p99 vs average latency.

Average hides the tail users feel. Always quote p99. Google / Uber.

99. How do you design spam/fraud checks on signup?

Rate limit, device signals, async review; do not block all on one ML call. Meta / Stripe.

100. Close an HLD: what do you say in the last minute?

Recap metric, biggest risk, what you would load-test. All.